

从XTP行情到XTPX行情API的变化

目录

- 从XTP行情到XTPX行情API的变化
 - 1. 结构体变动
 - 1.1. 结构体XTPMD变动
 - 1.2. 结构体XTPQuoteRebuildReq变动
 - 2. 已去除接口
 - 2.1. 去除期权逐笔行情以及订单簿行情相关的请求和回调接口
 - 2.2. 去除异步日志输出接口
 - 2.3. 去除回补行情登录、退出以及断线回调等相关接口
 - 2.4. 去除获取当前交易日接口
 - 2.5. 去除API部分参数设置接口
 - 3. 新增接口
 - 3.1. 新增接口SetConfigFile()
 - 3.2. 新增回调接口OnXTPQuoteNQFullInfo()
 - 3.3. 新增查询最新快照的相关接口
 - 3.4. 新增指数通行情的相关接口
 - 3.5. 新增港股通的行情的相关接口
 - 4. 功能变化接口
 - 4.1. 请求接口RequestRebuildQuote()的查询条件有变化
 - 4.2. 接口CreateQuoteApi()参数有变化

本文档介绍的XTPX API使用手册是在XTP API的基础上进行比较说明。

1. 结构体变动

1.1. 结构体XTPMD变动

快照回调接口OnDepthMarketData()里的结构体XTPMD参数有变化，XTP API版本参数如下：

```
///行情
typedef struct XTPMarketDataStruct
{
    // 代码
    ///交易所代码
    XTP_EXCHANGE_TYPE exchange_id;
    ///合约代码（不包含交易所信息），不带空格，以'\0'结尾
    char ticker[XTP_TICKER_LEN];

    // 价格
    ///最新价
```

```
double last_price;
///昨收盘
double pre_close_price;
///今开盘
double open_price;
///最高价
double high_price;
///最低价
double low_price;
///今收盘
double close_price;

// 期权数据
///昨日持仓量(张)(目前未填写)
int64_t pre_total_long_positon;
///持仓量(张)
int64_t total_long_positon;
///昨日结算价 (SH)
double pre_settl_price;
///今日结算价 (SH)
double settl_price;

// 涨跌停
///涨停价
double upper_limit_price;
///跌停价
double lower_limit_price;
///预留
double pre_delta;
///预留
double curr_delta;

/// 时间类，格式为YYYYMMDDHHMMSSsss
int64_t data_time;

// 量额数据
///数量，为总成交量（单位股，与交易所一致）
int64_t qty;
///成交金额，为总成交金额（单位元，与交易所一致）
double turnover;
///预留(无意义)
double avg_price;

// 买卖盘
///十档申买价
double bid[10];
///十档申卖价
double ask[10];
///十档申买量
int64_t bid_qty[10];
///十档申卖量
int64_t ask_qty[10];

// 额外数据
```

```

    ///成交笔数
    int64_t trades_count;
    ///当前交易状态说明
    char ticker_status[8];

    //对于新三板行情来说，以下结构和字段均无效
    ///数据
    union {
        XTPMarketDataStockExData  stk;
        XTPMarketDataOptionExData opt;
        XTPMarketDataBondExData  bond;
    };
    ///决定了union是哪种数据类型（2.2.32版本以前所用字段，仅为了保持兼容，不建议使用该字段）
    XTP_MARKETDATA_TYPE data_type;
    ///决定了union是哪种数据类型（2.2.32版本新增字段，更详细区分了行情快照数据类型）
    XTP_MARKETDATA_TYPE_V2 data_type_v2;
} XTPMD;

```

XTPX版本中结构体XTPMD跟XTP版本的相比较变化的地方:

- 去除结构体XTPMD中的参数data_type
上图中的参数**XTP_MARKETDATA_TYPE data_type不再使用**，请使用参数XTP_MARKETDATA_TYPE_V2 data_type_v2来代替。
- 结构体XTPMD中的参数值ticker_status有变化

XTP版本中将交易所传过来的交易状态及标志位转换过，XTPX版本直接透传交易所传过来的值，不再做转换。沪市如下：第0位：

- 'S',启动（开市前）时段
- 'C',开盘集合竞价时段
- 'T',连续竞价时段
- 'E',闭市时段
- 'P',产品停牌
- 'M',表示可恢复交易的熔断时段（盘中集合竞价）
- 'N',表示不可恢复交易的熔断时段（暂停交易至闭市）
- 'U',表示收盘集合竞价时段

第1位：

- '0',此产品不可正常交易
- '1',此产品可以正常交易
- 无意义填充格

第2位:

- '0',未上市
- '1',已上市

第3位:

- '0',此产品在当前时段，不接受进行新订单申报
- '1',此产品在当前时段，可接受进行新订单申报
- 无意义填充格

深市如下：第0位：

- 'S',启动（开市前）时段
- 'O',开盘集合竞价时段
- 'T',连续竞价时段
- 'B',休市
- 'C',表示收盘集合竞价时段
- 'E',闭市
- 'H',临时停牌
- 'A',盘后交易
- 'V',波动性中断

第1位：

- '0',正常状态
- '1',全天停牌

1.2. 结构体XTPQuoteRebuildReq变动

行情回补请求接口**RequestRebuildQuote()**中结构体**XTPQuoteRebuildReq**参数有变化，结构体原型如下：

```
///实时行情回补请求结构体
typedef struct XTPQuoteRebuildReq
{
    ///请求id 请求端自行管理定义
    int32_t request_id;
    ///请求数据类型 1-快照 2-逐笔
    XTP_QUOTE_REBUILD_DATA_TYPE data_type;
    ///请求市场 1-上海 2-深圳
    XTP_EXCHANGE_TYPE exchange_id;
    ///data_type=逐笔 表示逐笔通道号
    int16_t channel_number;
    ///预留
    char unuse[2];
    ///合约代码 以'\0'结尾 沪深A股6位 期权8位
    char ticker[16];
    ///data_type=逐笔 表示序列号begin；=快照 表示时间begin(格式为YYYYMMDDHHMMSSsss)
    =股票逐笔 表示时间begin(格式为YYYYMMDDHHMMSSsss)
    int64_t begin;
    ///data_type=逐笔 表示序列号end；=快照 表示时间end(格式为YYYYMMDDHHMMSSsss) =股票逐笔 表示时间end(格式为YYYYMMDDHHMMSSsss) 逐笔和股票逐笔区间：[begin, end]前后闭区间 快照区间：[begin, end) 前闭后开区间
    int64_t end;
}XTPQuoteRebuildReq;
```

上图中参数data_type跟XTP版本相比较新增了一个枚举值，data_type值分别描述如下：

```

///@brief XTP_QUOTE_DATA_TYPE是行情数据类型 逐笔，快照等
/////////////////////////////////////////////////////////////////
typedef uint32_t XTP_QUOTE_REBUILD_DATA_TYPE;
///<未知类型
constexpr uint32_t XTP_QUOTE_REBUILD_UNKNOW = 0;
///<快照类型
constexpr uint32_t XTP_QUOTE_REBUILD_MD = 1;
///<逐笔类型
constexpr uint32_t XTP_QUOTE_REBUILD_TBT = 2;
///<指定股票逐笔类型
constexpr uint32_t XTP_QUOTE_REBUILD_TICKER_TBT = 3;

```

- data_type=1，请求回补快照行情的类型，该请求回补类型依然保持了XTP版本的功能，即请求的是ticker在时间区间为[begin,end)的快照数据；
- data_type=2，请求的是channel_number在逐笔序列号seq区间为[begin,end]的逐笔数据，例如当按channel_number(频道代码)进行落盘行情数据时，发现在该频道代码中有缺失相关序列号的逐笔数据，可以选择data_type=2来请求回补行情；
- data_type=3，请求的是指定ticker在逐笔序号seq区间为[begin,end]的逐笔数据，例如当按ticker进行落盘行情数据时，发现该ticker的逐笔数据有缺失，并且只想回补该ticker的相关逐笔数据，不想要[begin,end]区间内的其它ticker的逐笔数据，可选择data_type=3请求回补逐笔行情。当指定ticker不知道自己所在的通道号channel_number时，可以不填channel_number，但后面再次查询的时候，需要根据查询结果填入正常的channel_number。此类型仅比data_type=2多了一个指定ticker过滤。

2. 已去除接口

2.1. 去除期权逐笔行情以及订单簿行情相关的请求和回调接口

由于期权只有快照行情，在XTP版本中的以下接口在XTPX版本中都已去除：

- SubscribeAllOptionTickByTick

```

///订阅全市场的期权逐笔行情（目前暂无此数据）
///@return 订阅全市场期权逐笔行情接口调用是否成功，“0”表示接口调用成功，非“0”表示接口调用出错
///@param exchange_id 表示当前全订阅的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，XTP_EXCHANGE_NQ表示新三板
///@remark 需要与全市场退订期权逐笔行情接口配套使用
virtual int SubscribeAllOptionTickByTick(XTP_EXCHANGE_TYPE exchange_id = XTP_EXCHANGE_UNKNOWN) = 0;

```

- UnSubscribeAllOptionTickByTick

```

///退订全市场的期权逐笔行情（目前暂无此数据）
///@return 退订全市场期权逐笔行情接口调用是否成功，“0”表示接口调用成功，非“0”表示接口调用出错
///@param exchange_id 表示当前退订的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，XTP_EXCHANGE_NQ表示新三板
///@remark 需要与订阅全市场期权逐笔行情接口配套使用
virtual int UnSubscribeAllOptionTickByTick(XTP_EXCHANGE_TYPE exchange_id = XTP_EXCHANGE_UNKNOWN) = 0;

```

- SubscribeAllOptionOrderBook

```

///订阅全市场的期权行情订单簿（目前暂无此数据）
///@return 订阅全市场期权行情订单簿接口调用是否成功，“0”表示接口调用成功，非“0”表示接口调用出错
///@param exchange_id 表示当前全订阅的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，XTP_EXCHANGE_NQ表示新三板
///@remark 需要与全市场退订期权行情订单簿接口配套使用
virtual int SubscribeAllOptionOrderBook(XTP_EXCHANGE_TYPE exchange_id = XTP_EXCHANGE_UNKNOWN) = 0;

```

- UnSubscribeAllOptionOrderBook

```

///退订全市场的期权行情订单簿（目前暂无此数据）
///@return 退订全市场期权行情订单簿接口调用是否成功，“0”表示接口调用成功，非“0”表示接口调用出错
///@param exchange_id 表示当前退订的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，XTP_EXCHANGE_NQ表示新三板
///@remark 需要与订阅全市场期权行情订单簿接口配套使用
virtual int UnSubscribeAllOptionOrderBook(XTP_EXCHANGE_TYPE exchange_id = XTP_EXCHANGE_UNKNOWN) = 0;

```

- OnSubscribeAllOptionTickByTick

```

///订阅全市场的期权逐笔行情应答
///@param exchange_id 表示当前全订阅的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，XTP_EXCHANGE_NQ表示新三板
///@param error_info 取消订阅合约时发生错误时返回的错误信息，当error_info为空，或者error_info.error_id为0时，表明没有错误
///@remark 需要快速返回
virtual void OnSubscribeAllOptionTickByTick(XTP_EXCHANGE_TYPE exchange_id, XTPRI *error_info) {};

```

- OnUnSubscribeAllOptionTickByTick

```

///退订全市场的期权逐笔行情应答
///@param exchange_id 表示当前退订的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场
（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，
XTP_EXCHANGE_NQ表示新三板
///@param error_info 取消订阅合约时发生错误时返回的错误信息，当error_info为空，或者
error_info.error_id为0时，表明没有错误
///@remark 需要快速返回
virtual void OnUnSubscribeAllOptionTickByTick(XTP_EXCHANGE_TYPE exchange_id, XTPRI
*error_info) {};

```

- OnSubscribeAllOptionOrderBook

```

///订阅全市场的期权行情订单簿应答
///@param exchange_id 表示当前全订阅的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场
（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，
XTP_EXCHANGE_NQ表示新三板
///@param error_info 取消订阅合约时发生错误时返回的错误信息，当error_info为空，或者
error_info.error_id为0时，表明没有错误
///@remark 需要快速返回
virtual void OnSubscribeAllOptionOrderBook(XTP_EXCHANGE_TYPE exchange_id, XTPRI
*error_info) {};

```

- OnUnSubscribeAllOptionOrderBook

```

///退订全市场的期权行情订单簿应答
///@param exchange_id 表示当前退订的市场，如果为XTP_EXCHANGE_UNKNOWN，表示沪深全市场
（不包括新三板），XTP_EXCHANGE_SH表示为上海全市场，XTP_EXCHANGE_SZ表示为深圳全市场，
XTP_EXCHANGE_NQ表示新三板
///@param error_info 取消订阅合约时发生错误时返回的错误信息，当error_info为空，或者
error_info.error_id为0时，表明没有错误
///@remark 需要快速返回
virtual void OnUnSubscribeAllOptionOrderBook(XTP_EXCHANGE_TYPE exchange_id, XTPRI
*error_info) {};

```

2.2. 去除异步日志输出接口

XTP版本中有控制异步日志输出的接口SetUDPSeqLogOutPutFlag()，接口描述如下：

```

///设定UDP收行情时是否输出异步日志
///@param flag 是否输出标识，默认为true，如果不想输出“udpseq”开头的异步日志，请设置此参
数为false
///@remark 此函数可不调用，如果调用则必须在Login之前调用，否则不会生效
virtual void SetUDPSeqLogOutPutFlag(bool flag = true) = 0;

```

该接口在XTPX版本不再提供，控制异步日志的输出是以参数形式放到了接口CreateQuoteApi()中，具体可以参考下文4.1 章节中的说明。

2.3. 去除回补行情登录、退出以及断线回调等相关接口

行情回补在XTPX版本中不再需要登录行情回补服务器，在行情缺失时，可直接调用请求接口RequestRebuildQuote，所以在XTP版本中使用的以下接口在XTPX版本中都已去除：

- LoginToRebuildQuoteServer

```
///用户登录回补服务器请求
///@return 登录是否成功，“0”表示登录成功，“-1”表示连接服务器出错，此时用户可以调用
GetApiLastError()来获取错误代码，“-2”表示已存在连接，不允许重复登录，如果需要重连，请先
logout，“-3”表示输入有错误
///@param ip 服务器ip地址，类似“127.0.0.1”
///@param port 服务器端口号
///@param user 登陆用户名
///@param password 登陆密码
///@param sock_type “1”代表TCP，“2”代表UDP
///@param local_ip 本地网卡地址，类似“127.0.0.1”
///@remark 此函数为同步阻塞式，不需要异步等待登录成功，当函数返回即可进行后续操作，此api
只能有一个连接。回补服务器会在无消息交互后定时断线，请注意仅在需要回补数据时才保持连接，回
补完成后请及时logout
virtual int LoginToRebuildQuoteServer(const char* ip, int port, const char* user,
const char* password, XTP_PROTOCOL_TYPE sock_type, const char* local_ip = NULL) =
0;
```

- LogoutFromRebuildQuoteServer

```
///登出回补服务器请求
///@return 登出是否成功，“0”表示登出成功，非“0”表示登出出错，此时用户可以调用
GetApiLastError()来获取错误代码
///@remark 此函数为同步阻塞式，不需要异步等待登出，当函数返回即可进行后续操作
virtual int LogoutFromRebuildQuoteServer() = 0;
```

- OnRebuildQuoteServerDisconnected

```
///当客户端与回补行情服务器通信连接断开时，该方法被调用。
///@param reason 错误原因，请与错误代码表对应
///@remark api不会自动重连，当断线发生时，请用户自行选择后续操作。回补服务器会在无消息交
互后会定时断线，请注意仅在需要回补数据时才保持连接，无回补需求时，无需登陆。
virtual void OnRebuildQuoteServerDisconnected(int reason) {};
```

2.4. 去除获取当前交易日接口

XTP版本中获取当前交易日接口GetTradingDay()在XTPX版本里已去除，不再使用，对应XTP版本的函数原型如下：

```
///获取当前交易日
///@return 获取到的交易日
///@remark 只有登录成功后,才能得到正确的交易日
virtual const char *GetTradingDay() = 0;
```

2.5. 去除API部分参数设置接口

XTP版本中的接口 SetUDPBufferSize()、SetUDPRecvThreadAffinity()、SetUDPRecvThreadAffinityArray()、SetUDPParseThreadAffinity()、SetUDPParseThreadAffinityArray()在XTPX版本里都已去除，统一以参数形式放入到配置文件中设置。配置文件的说明见[下文3.1 章节](#)中的说明。

3. 新增接口

3.1. 新增接口SetConfigFile()

在XTPX版本中，登录udp连接方式的行情服务器之前，必须先调用接口SetConfigFile正确设置配置文件，否则无法获取到行情数据，接口SetConfigFile()描述如下：

```
///设置行情接收的配置文件
///@return 设置配置文件是否成功，true-成功，false-失败，需要检查配置文件是否正确
///@param filename 包含绝对路径的配置文件名
///@remark 此函数必须在Login之前调用，如果不调用，将无法获取行情
virtual bool SetConfigFile(const char* filename) = 0;
```

该接口设置的配置文件是有关行情订阅的参数设置，参数格式如下：

```
[md]
decode_flag = 1 #1表示解码的快照数据，目前api提供的只有解码的行情数据
parse_cpu_id = 2 #解析线程绑核的cpu核id(逻辑核)，0表示不绑核
[md.normal]
enable = ON #ON 表示启用软件行情的快照，OFF表示不启用
local_ip = 127.0.0.1 #接收快照所在组播组的网段的网卡地址
recv_cpu_id = 3 #接收线程绑核的cpu核id(逻辑核)，0表示不绑核
enable_efvi = OFF #基于solarflare网卡的一种api标识
L1_buf_capacity = 256 #一级缓存的大小，最小为256k个缓存单元
L2_buf_capacity = 8 #二级缓存的大小，最小为8k个缓存单元
[md.fpga]
enable = OFF #ON表示启用硬件行情的快照，OFF表示不启用
local_ip = 127.0.0.1 #接收快照所在组播组的网段的网卡地址
recv_cpu_id = 3 #接收线程绑核的cpu核id(逻辑核)，0表示不绑核

enable_efvi = OFF #基于solarflare网卡的一种api标识
L1_buf_capacity = 256 #一级缓存的大小，最小为256k个缓存单元
L2_buf_capacity = 8 #二级缓存的大小，最小为8k个缓存单元
```

```
[tbt]
decode_flag = 1    #1表示解码的逐笔数据，目前api提供的只有解码的行情数据
parse_cpu_id = 4   #解析线程绑核的cpu核id(逻辑核)，0表示不绑核
[tbt.normal]
enable = ON    #ON表示启用软件行情的逐笔，OFF表示不启用
local_ip = 127.0.0.1 #接收逐笔数据所在组播组的网段的网卡地址
recv_cpu_id = 5   #接收线程绑核的cpu核id(逻辑核)，0表示不绑核
enable_efvi = OFF #基于solarflare网卡的一种api标识
L1_buf_capacity = 256 #一级缓存的大小，最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小，最小为8k个缓存单元
[tbt.fpga]
enable = OFF    #ON表示启用硬件行情的逐笔，OFF表示不启用
local_ip = 127.0.0.1 #接收逐笔数据所在组播组的网段的网卡地址
recv_cpu_id = 5   #接收线程绑核的cpu核id(逻辑核)，0表示不绑核
enable_efvi = OFF #基于solarflare网卡的一种api标识
L1_buf_capacity = 256 #一级缓存的大小，最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小，最小为8k个缓存单元

[ob]
decode_flag = 1    #1表示解码的订单簿数据，目前api提供的只有解码的行情数据。
parse_cpu_id = 6    #解析线程绑核的cpu核id(逻辑核)，0表示不绑核
[ob.normal]
enable = ON    #ON表示启用软件行情的订单簿，OFF表示不启用
local_ip = 127.0.0.1 #接收订单簿所在组播组的网段的网卡地址
recv_cpu_id = 7     #接收线程绑核的cpu核id(逻辑核)，0表示不绑核
enable_efvi = OFF   #基于solarflare网卡的一种api标识
L1_buf_capacity = 256 #一级缓存的大小，最小为256k个缓存单元
L2_buf_capacity = 8  #二级缓存的大小，最小为8k个缓存单元

[idxpress]
decode_flag = 1
parse_cpu_id = 10
[idxpress.normal]
enable = ON
local_ip = 127.0.0.1
recv_cpu_id = 11
enable_efvi = OFF
L1_buf_capacity = 256
L2_buf_capacity = 8

[hkc]
decode_flag = 1
parse_cpu_id = 8
[hkc.normal]
enable = ON
local_ip = 127.0.0.1
recv_cpu_id = 9
enable_efvi = OFF
L1_buf_capacity = 256
L2_buf_capacity = 8

[subscribe_quote_type]
sh_ob = OFF    #上海订单簿行情是否打开标志，OFF:关闭，ON：打开
```

```

sz_ob = OFF          #深圳订单簿行情是否打开标志，OFF:关闭，ON：打开
sh_level1_md_index = ON #L1沪市指数快照行情是否打开标识
sh_level1_md_stock = ON #L1 沪市股票快照行情是否打开标识
sh_level1_md_option = ON #L1 沪市期权快照行情是否打开标识
sh_level2_md_index = ON #L2 沪市指数快照行情是否打开标识
sh_level2_md_stock = ON #L2 沪市股票快照行情是否打开标识
sh_level2_md_bond = ON #L2 沪市债券快照行情是否打开标识
sh_level2_tbt_stock = ON #L2 沪市股票逐笔行情是否打开标识
sh_level2_tbt_bond = ON #L2 沪市债券逐笔行情是否打开标识
sz_level1_md_index = ON #L1 深市指数快照行情是否打开标识
sz_level1_md_stock = ON #L1 深市股票快照行情是否打开标识
sz_level1_md_option = ON #L1 深市期权快照行情是否打开标识
sz_level1_md_bond = ON #L1 深市债券行情是否打开标识
sz_level2_md_index = ON #L2 深市指数快照行情是否打开标识
sz_level2_md_stock = ON #L2 深市股票快照行情是否打开标识
sz_level2_md_bond = ON #L2 深市债券快照行情是否打开标识
sz_level2_tbt_stock = ON #L2 深市股票逐笔行情是否打开标识
sz_level2_tbt_bond = ON #L2 深市债券逐笔行情是否打开标识
nq_rawtxt = OFF      #新三板指数通行情是否打开标识
nq_md_bond = OFF      #新三板债券快照行情是否打开标识
nq_tbt_bond = OFF     #新三板债券逐笔行情是否打开标识
#L1沪市指数通行情是否打开标识
sh_level1_rawtxt = ON
#以下为港股通hkc相关订阅配置
sz_level1_md_hkc = ON
sz_level1_md_hkcsta = ON

```

配置文件使用说明：

- 对于md、tbt这两种行情，软件行情和硬件行情接收方式都设置启用的话，即对应行情下面 **enable = ON**，这两路行情数据接收方式，API会自动过滤出更快的行情数据并推送给用户，所以用户只会收到一份行情数据，不会收到两份同样的行情数据。
- md、tbt这两种行情，每种行情都有两路行情接收方式，即软件行情和硬件行情，对应两个接收线程，只有 **enable = ON** 才会启动对应的接收线程。ob行情只有软件行情的ob数据，对应一个接收线程，同样 **enable = ON** 才会启动对应的接收线程。如果接收线程绑核的话，两路行情接收方式的recv_cpu_id可以设定不同的cpu核id，也可以是相同的cpu核id，设置为0表示不绑核。
- md、tbt、ob这三种行情，分别对应三个解析线程，只有对应种类的行情下面其中一个 **enable = ON** 就会启动该解析线程。如果解析线程绑核的话，三种行情的parse_cpu_id分别设定不同的cpu核id，如果设置为0，表示不绑核。
- parse_cpu_id和recv_cpu_id设置绑核的cpu，如绑定cpu第二个逻辑核，设置为1，建议绑定后面的逻辑核。
- L1_buf_capacity和L2_buf_capacity缓存单元的大小跟数据类型有关。

3.2. 新增回调接口OnXTPQuoteNQFullInfo()

当订阅了三板快照后会自动推送相关合约的静态数据，接口描述如下：

```

///新三板合约完整静态信息盘中推送通知
///@param ticker_info 合约完整静态信息
virtual void OnXTPQuoteNQFullInfo(XTPNQFI* ticker_info) { (void)ticker_info; };

```

3.3. 新增查询最新快照的相关接口

获取合约的最新快照信息查询接口QueryTickersLatestMarketData()以及对应的回调响应接口OnQueryTickersLatestMarketData()，描述如下：

```

///获取合约的最新快照信息
///@return 发送查询请求是否成功，“0”表示发送查询请求成功，非“0”表示发送查询请求不成功
///@param ticker 合约ID数组，注意合约代码必须以'\0'结尾，不包含空格
///@param count 要查询的合约个数
///@param exchange_id 交易所代码
virtual int QueryTickersLatestMarketData(char *ticker[], int count,
XTP_EXCHANGE_TYPE exchange_id) = 0;

///查询合约的最新快照信息应答
///@param market_data 合约的最新快照信息
// ///@param error_info 查询合约的最新快照信息时发生错误时返回的错误信息，当error_info
为空，或者error_info.error_id为0时，表明没有错误
///@param is_last 是否此次查询的最后一个应答，当为最后一个的时候为true，如果为false，表
示还有其他后续消息响应
virtual void OnQueryTickersLatestMarketData(XTPMD* market_data, XTPRI *error_info,
bool is_last) { (void)market_data; (void)error_info; (void)is_last; };

```

3.4. 新增指数通行情的相关接口

XTPX新增了指数的行情功能，如果用户需要订阅，配置文件ConfigFile里涉及到的上海指数行情参数如下：

```

.....
[idxpress]
decode_flag = 1
parse_cpu_id = 10
[idxpress.normal]
enable = ON
local_ip = 127.0.0.1
recv_cpu_id = 11
enable_efvi = OFF
L1_buf_capacity = 256
L2_buf_capacity = 8
[subscribe_quote_type]
#L1沪市指数行情是否打开标识
sh_level1_rawtxt = ON
.....

```

用户配置好相关参数后，再调用订阅接口，指数行情相关接口详情如下：

- SubscribeAllIndexPress

```

    ///订阅指数通行情
    ///@return 发送订阅请求是否成功，“0”表示发送订阅请求成功，非“0”表示发送订阅
    请求不成功
    ///@remark 对应响应函数是OnSubscribeAllIndexPress()
    virtual int SubscribeAllIndexPress() = 0;

```

- UnSubscribeAllIndexPress

```

    ///取消订阅指数通行情
    ///@return 发送订阅请求是否成功，“0”表示发送订阅请求成功，非“0”表示发送订阅
    请求不成功
    ///@remark 对应响应函数是OnSubscribeAllIndexPress()
    virtual int UnSubscribeAllIndexPress() = 0;

```

- OnSubscribeAllIndexPress

```

    ///订阅指数通行情应答
    ///@param error_info 订阅指数通时发生错误时返回的错误信息，当error_info为
    空，或者error_info.error_id为0时，表明没有错误
    ///@remark 需要快速返回
    virtual void OnSubscribeAllIndexPress(XTPRI *error_info) {
    (void)error_info; };

```

- OnUnSubscribeAllIndexPress

```

    ///取消订阅指数通行情应答
    ///@param error_info 取消订阅指数通时发生错误时返回的错误信息，当
    error_info为空，或者error_info.error_id为0时，表明没有错误
    ///@remark 需要快速返回
    virtual void OnUnSubscribeAllIndexPress(XTPRI *error_info) {
    (void)error_info; };

```

- OnIndexPress

```

    ///指数通行情通知
    ///@param idx 指数通的行情数据
    ///@remark 订阅指数通行情的时候触发推送通知
    virtual void OnIndexPress(XTPIndexPress *idx) { (void)idx; };

    ///上海指数通快照行情
    typedef struct XTPIndexPress {

```

```

// 代码
///合约代码（不包含交易所信息），不带空格，以'\0'结尾
char    ticker[XTP_QUOTE_TICKER_LEN];
///交易所代码(无意义)
XTP_EXCHANGE_TYPE exchange_id;
///数据来源，7=指数通。
char    data_source;
char    unused[3];

/// 时间类，格式为YYYYMMDDHHMMSSsss
int64_t data_time;

// 价格
///最新价
double  last_price;
///昨收盘
double  pre_close_price;
///今开盘
double  open_price;
///最高价
double  high_price;
///最低价
double  low_price;
///今收盘
double  close_price;

// 量额数据
///总成交量
int64_t qty;
///总成交金额
double  turnover;
///成交笔数
int64_t trades_count;
}XTPIP;

```

3.5. 新增港股通的行情的相关接口

XTPX新增了港股通行情功能，如果用户需要订阅，配置文件ConfigFile里港股通行情的相关参数如下：

```

.....
[hkc]
decode_flag = 1
parse_cpu_id = 8
[hkc.normal]
enable = ON
local_ip = 127.0.0.1
recv_cpu_id = 9
enable_efvi = OFF
L1_buf_capacity = 256

```

```
L2_buf_capacity = 8
[subscribe_quote_type]
#以下为港股通hkc相关订阅配置
sz_level1_md_hkc = ON
sz_level1_md_hkcsta = ON
.....
```

用户配置好相关参数后，再调用订阅接口，港股通行情相关接口如下：

- **SubscribeAllHKCMarketData**

```
///订阅港股通的行情
///@return 发送订阅请求是否成功，“0”表示发送订阅请求成功，非“0”表示发送订
请求不成功
///@remark 对应响应函数是OnSubscribeAllHKCMarketData()
virtual int SubscribeAllHKCMarketData() = 0;
```

- **UnSubscribeAllHKCMarketData**

```
///取消订阅港股通的行情
///@return 发送订阅请求是否成功，“0”表示发送订阅请求成功，非“0”表示发送订
请求不成功
///@remark 对应响应函数是OnSubscribeAllHKCMarketData()
virtual int UnSubscribeAllHKCMarketData() = 0;
```

- **OnSubscribeAllHKCMarketData**

```
///订阅港股通行情应答
///@param error_info 订阅港股通时发生错误时返回的错误信息，当error_info
为空，或者error_info.error_id为0时，表明没有错误
///@remark 需要快速返回
virtual void OnSubscribeAllHKCMarketData(XTPRI *error_info) {
(void)error_info; };
```

- **OnUnSubscribeAllHKCMarketData**

```
///取消订阅港股通行情应答
///@param error_info 取消订阅港股通时发生错误时返回的错误信息，当
error_info为空，或者error_info.error_id为0时，表明没有错误
///@remark 需要快速返回
virtual void OnUnSubscribeAllHKCMarketData(XTPRI *error_info) {
(void)error_info; };
```

- OnHKRLData

```

///港股通实时额度通知
///@param hkc_data 港股通实时额度数据
///@remark 订阅港股通行行情的时候会触发推送通知
virtual void OnHKRLData(XTPHKCRL *hkc_data) { (void)hkc_data; };

///港股通实时额度
typedef struct XTPHKCRealtimeLimit {
    /// 时间类，格式为YYYYMMDDHHMMSSsss
    int64_t data_time;

    /// 市场代码
    char market_id[8];
    /// 市场板块代码, 预留
    char market_segment_id[8];

    /// 每日初始额度
    int64_t threshold_amount;
    /// 日中剩余额度
    int64_t pos_amt;
    /// 额度状态:
    /// 1=额度不可用; pos_amt = 0.0000
    /// 2=额度可用(额度可用且低于当日初始额度的30%时); pos_amt = 当前实际剩余
    /// 3=额度充足(额度可用且大于或等于当日初始额度的30%时); pos_amt = 0.0000
    char amount_status;
    char unused[7];
} XTPHKCRL;

```

- OnHKCMarketData

```

///港股通行行情通知
///@param hkc_md 港股通的行情数据
///@remark 订阅港股通行行情的时候会触发推送通知
virtual void OnHKCMarketData(XTPHKCMD *hkc_md) { (void)hkc_md; };

///深交所港股通快照行情
typedef struct XTPHKCMarketdata {
    /// 代码
    ///合约代码(不包含交易所信息)，不带空格，以'\0'结尾
    char ticker[XTP_QUOTE_TICKER_LEN];
    ///交易所代码
    XTP_EXCHANGE_TYPE exchange_id;
    ///预留
    int32_t unused;

    /// 时间类，格式为YYYYMMDDHHMMSSsss
    int64_t data_time;
    ///当前交易状态说明

```



```
char    ticker_status[8];

///成交笔数
int64_t trades_count;
///数量，为总成交量（单位股，与交易所一致）
int64_t qty;
///成交金额，为总成交金额（单位元，与交易所一致）
double  turnover;

///涨停价
double  upper_limit_price;
///跌停价
double  lower_limit_price;

// 价格
///最新价
double  last_price;
///昨收盘
double  pre_close_price;
///参考价
double  ref_price;
///最高价
double  high_price;
///最低价
double  low_price;
///按盘价/盘后为今收价
double  nominal_price;

///买盘上限价 如果处于开盘集合竞价时段，则表示开盘竞价时段的买盘价格上限
double  bid_upper_price;
///买盘下限价 如果处于开盘集合竞价时段，则表示开盘竞价时段的买盘价格下限
double  bid_lower_price;
///卖盘上限价 如果处于开盘集合竞价时段，则表示开盘竞价时段的卖盘价格上限
double  ask_upper_price;
///卖盘下限价 如果处于开盘集合竞价时段，则表示开盘竞价时段的卖盘价格下限
double  ask_lower_price;

///冷静期开始时间 大于0表示当前处于触发VCM的冷静期
int64_t complex_event_start_time;
///冷静期结束时间 大于0表示当前处于触发VCM的冷静期
int64_t complex_event_end_time;

// 买卖盘
///十档申买价
double  bid[10];
///十档申卖价
double  ask[10];
///十档申买量
int64_t bid_qty[10];
///十档申卖量
int64_t ask_qty[10];
} XTPHKCMD;
```

深交所港股通行情字段**ticker_status**表示交易状态，标志位如下：第0位：

- 'S',启动（开市前）时段
- 'O',开盘集合竞价时段
- 'T',连续竞价时段
- 'B',休市
- 'C',表示收盘集合竞价时段
- 'E',闭市
- 'H',临时停牌

4. 功能变化接口

4.1. 请求接口RequestRebuildQuote()的查询条件有变化

回补行情的请求接口RequestRebuildQuote()在XTPX版本中查询条件有所变化，逐笔行情回补功能新增了根据ticker条件回补的类型。例如从头开始请求回补股票000001.sz的逐笔行情代码示例：

```
//回补指定股票的逐笔行情
XTPX::API::XTPQuoteRebuildReq req;
memset(&req, 0, sizeof(XTPX::API::XTPQuoteRebuildReq));
req.request_id = 1;
req.exchange_id = XTPX::API::XTP_EXCHANGE_SZ;
strcpy(req.ticker, "000001");
req.data_type = XTPX::API::XTP_QUOTE_REBUILD_TICKER_TBT;
req.channel_number = 0; //第一次从头开始查可以填0，后面再次查询的时候，需要
根据查询结果填入正常的channel_number
req.begin = 0; //第一次不知道seq区间，可填0
req.end = 1000000; //第一次不知道seq区间，可随意填一个超大值
int ret = pQuoteApi->RequestRebuildQuote(&req);
```

此时一次回补的行情并不是指定seq区间的所有000001.sz逐笔行情，可能只有部分数据，需要客户根据回补结果视情况看是否需要再次发起后续查询。

4.2. 接口CreateQuoteApi()参数有变化

XTPX版本中创建QuoteApi的接口CreateQuoteApi()增加了一个控制异步日志输出的参数，默认为true，表示开启异步日志，接口CreateQuoteApi()描述如下：

```
///创建QuoteApi
///@param client_id （必须输入）用于区分同一用户的不同客户端，由用户自定义
///@param save_file_path （必须输入）存贮订阅信息文件的目录，请设定一个有可写权限的真实
存在的路径，如果路径不存在的话，可能会因为写冲突而造成断线
///@param log_level 日志输出级别
///@param udpseq_output udpseq异步日志是否输出标识，默认为true，如果不想输出异步日志，
请设置此参数为false
///@return 创建出的UserApi
///@remark 如果一个账户需要在多个客户端登录，请使用不同的client_id，系统允许一个账户同时
登录多个客户端，但是对于同一账户，相同的client_id只能保持一个session连接，后面的登录在前
一个session存续期间，无法连接
```

```
static QuoteApi *CreateQuoteApi(uint8_t client_id, const char *save_file_path,  
XTP_LOG_LEVEL log_level = XTP_LOG_LEVEL_DEBUG, bool udpseq_output = true);
```